

Slice 15 — Making the Override Reachable, and Deciding What Happens After It

Date: 3 September 2026 **Branch:** `slice-15-oversold-override-behaviour` **Base:** `main` at `31e88ca1` (Slice 14)

1. What this slice was asked to do

Three things came out of the owner's testing of Slice 14 at the counter. The stock limits themselves were confirmed working, and the bug that persisted the cart and the member after a completed sale was confirmed fixed. What remained was that the owner could not find the "Sell anyway" override anywhere on the register screen, and asked whether something was missing on the oversold bug.

Investigation of the source confirmed the override was genuinely unreachable, and that this was a design flaw in Slice 14 rather than a matter of the owner not finding a control that was there. The override button existed only inside a panel rendered when `stockBlocks.length > 0`, and the Phase-2 clamp introduced in the same slice prevents a cart from ever reaching that state through normal use. The refusal message, meanwhile, ended with the sentence `Use "Sell anyway" if the unit is on the shelf.` — an instruction pointing at a button that could not be displayed. The owner approved Fix A to move the override into the refusal itself, and option 3 of Fix B to surface oversold sales both as a back-office list and as a per-product flag.

The third item was a question the owner deliberately did not answer from intuition. He asked for research into the enterprise industry standard for what a register should do at the end of a sale that used an override, so the behaviour would not be reinvented. That research is recorded in full, with sources, in `docs/slice-15-research-oversold-industry-standard.md`, and this document describes only what was built from it.

2. The answer the research produced

The research consulted four independent authorities: Lightspeed Retail's own documentation on its Negative Inventory report, Shopify's Help Center and Shopify POS inventory documentation, Oracle NetSuite's published cycle-counting guidance, and the Washington Administrative Code sections binding on this licence. It produced a consistent five-part answer.

The sale completes and is never blocked. Shopify states this explicitly, noting that the "Continue selling when out of stock" setting "doesn't apply to orders placed from Shopify POS," that "staff can continue selling products when available inventory reaches zero and below," and that "POS warns staff before they sell an item that's not available." The warning belongs before the sale, at the moment of adding the item, not after it. Lightspeed places its insufficient-stock alert in the same position.

Nothing about the discrepancy goes on the customer's receipt. No source examined places it there, because it is an internal inventory-control record and not information the customer transacted for.

The variance is recorded with attribution. Lightspeed's Negative Inventory report columns define the minimum useful record: the item, the quantity on hand, the

adjustment reason, the source sale, the quantity removed, the employee, and the date and time. A log without attribution is not a control.

The product is placed on a persistent back-office working list, which is what the owner had independently asked for as Fix B option 3.

And an oversell triggers a count. NetSuite names this pattern "Opportunity-based" cycle counting, describing "exception-based cycle counts, such as when the stock goes below its predetermined threshold, or when short-picks occur." An oversell at a register is precisely a short-pick. Their "zero count" practice — where emptying a bin causes "a command given to the warehouse worker to have them count the bin and confirm it is empty" — is the direct precedent for prompting the person standing at the counter, at the moment of discovery, while they are in front of the product.

Washington law then settles the one question the vendors leave open. WAC 314-55-087(2) governs record keeping inside a point-of-sale system and sets two tests: that it "provides an audit trail so that details ... underlying the summary accounting data may be identified and made available upon request," and that it "provides the opportunity to trace any transaction back to the original source or forward to a final total." This is stricter than vendor convention and binding on this store, and it is exactly where a naive system loses the trail: the existing decrement clamps the stored level at zero, which is correct because a negative on-hand figure cannot be reported, but the clamp destroys the size of the discrepancy. Once a level is pinned at zero there is no way to recover "sold two against one tracked" from the level alone.

3. What was built

Fix A — the override now travels with the refusal

The dead-end instruction string was removed. Both places where the register can refuse to add a unit — the manual/scan path at `SaleFlow.tsx:2250` and the cart's `+` control at `SaleFlow.tsx:2740` — now populate a `stockOverrideOffer` state (`:2217`) carrying the refusal reason. A new `StockOverrideOffer` component (`:5160`) renders that reason, the menu's last-refreshed age, and a "Sell anyway — the unit is on the shelf" button, and it is rendered at both refusal sites (`:2564` , `:3242`).

The override remains open to any staff member, with no PIN, per the owner's decision in the previous round. Scan-required and price overrides remain manager-locked and are untouched. This is verified by a test that reads the component body itself and asserts the absence of any approval machinery, rather than trusting the intent.

The post-sale count command

At the moment the sale is enqueued (`SaleFlow.tsx:827`), while the cart is still intact, the register computes the variance and sets a count prompt. The "Count check needed" panel (`SaleFlow.tsx:920`) then appears on the sale-complete screen above an unconditional "Done — lock register" button, so it informs without gating, matching finding one. It names the product and states the arithmetic, and it is deliberately absent from the receipt.

The variance is driven by the numbers rather than by the override toggle, which matters in both directions. A budtender can flip the override on and then sell nothing past the count, which is not a variance and must not raise a count task; conversely, a menu refresh that lowers a count underneath an existing cart is a real variance even though nobody touched the switch.

Fix B option 3 — the back-office list and the per-product flag

A "Stock needing recount" section was added to the cycle-counts page (`admin/inventory/cycle-counts/page.tsx:167`), fed by `loadOversoldReport` (`:96`). It shows a per-product roll-up — how many units the count is off by, across how many sales, and when it was last seen — with the per-sale detail beneath it and a link to each source sale. It reports only; nothing on the list can be adjusted from there, because fixing a count is still a count.

4. Two decisions that changed during the build

Both are recorded because both went against the plan, and in each case the code proved the plan wrong.

The first concerns where the durable record is written. The original plan had the register write an attributed audit record and queue it through the offline pipeline. Attempting it surfaced a type error: the event type did not exist in `POS_EVENT_TYPES`, and adding one would require a migration and a server-side processor. Investigating that led to the discovery that the record already exists. The server detects the same shortfall during the decrement, against the live inventory level rather than a cached one, and stamps it onto the sale's own `order_events` row. Writing a second copy from the register would have produced two records of a single event that could disagree, with the register's being the less trustworthy of the two — the opposite of the audit trail the regulation requires. The client-side write was removed, and when the resulting helper turned out to have no remaining production caller it was deleted rather than left in place, since unused code implying the register writes audit rows would mislead the next reader.

The second concerns a promise made to staff. A draft of the count panel told the budtender the shortfall had been "logged with your name, this register and the sale." Checking that claim against the code showed it was false: the decrement runs server-side and stamps `actor_label` as `"system"`, with the employee reachable through the sale itself rather than through that field. The wording was narrowed to what the system actually guarantees. The same discovery led to suppressing the literal string `system` in the back-office list, where it would have been displayed beside each sale and read as a person's name, implying an attribution the row does not carry. The trace to the responsible employee is the link to the sale, which is what the regulation asks for.

5. Verification

The two new pure cores carry their own embedded self-tests, mirrored by vitest files following the Slice 14 precedent. `oversold-variance-core` includes a nested-loop invariant asserting that anything the stock ceiling permits can never register as a variance, which prevents the register nagging for a count after a legal sale. `oversold-report-core` is tested by round-tripping against the real note writer — the tests build an actual oversold plan with `buildVariantDecrementPlan` and parse the output of `summarizeDecrement`, so the reader and writer cannot drift apart silently. That matters because a parser that quietly finds nothing produces an empty list, and an empty list looks exactly like "no problems."

The wiring assertions were demonstrated failing against the pre-fix code before the fix was written, in keeping with the standing red-first rule. Two assertions were subsequently rewritten, and neither was loosened: one was made stricter, reading the override component's own body instead of a loose window of surrounding characters that had swept in unrelated markup and produced a false report of a PIN gate; the other was replaced with an assertion of the real architecture after the client-side write path was removed by design. The report parser's own tests also caught a

genuine bug during development, where splitting the note on the quote character cut each entry in half at its closing quote.

Full suite: 528 files, 13,426 tests, all passing, against 526 files and 13,387 tests at Slice 14 — exactly the two new files and their assertions, with no regressions. `tsc --noEmit` clean and ESLint clean on all touched files.

6. What the owner should check at the counter

Add a product the register believes is out of stock, or push a line past its count. The refusal now appears with a "Sell anyway — the unit is on the shelf" button directly inside it, along with a note of when the menu was last refreshed. Any staff member can use it; no PIN is required. Complete the sale as normal — it will not be blocked, and the customer's receipt will show nothing about the discrepancy. The sale-complete screen will then show "Count check needed" naming the product and the amount the count is off by, above the usual lock button, which remains enabled. The product will also appear under "Stock needing recount" on the cycle-counts page in the back office, with a link back to the sale it came from.